Université du Québec à Chicoutimi

LIARA - Laboratoire sur l'Intelligence Ambiante pour la Reconnaissance d'Activité

Stage - Juin à Décembre 2026

Projection des données Unitree LiDAR L2

Étudiant
BONNEAU Mathieu

Remis à
Julien MAITRE

21 juillet 2026

UQAC
Université du Québec
à Chicoutimi

Table des matières

1 Objectif et chaîne de traitement **2**

2 Références méthodologiques **2**

3 Projection sphérique **2**
3.1 Coordonnées sphériques 2
3.2 Passage aux pixels 3
3.3 Champ de vision 3

4 Premiers résultats **3**
4.1 Projection d'un seul nuage 3
4.2 Origine des bandes noires 4

5 Améliorations apportées **4**
5.1 Accumulation de plusieurs acquisitions 4
5.2 Choix d'une image panoramique 5
5.3 Fusion optimisée des images 6

6 Commandes nécessaires **6**
6.1 Dépendances Python 6
6.2 Compilation du paquet ROS 2 Unitree 6
6.3 Démarrage du lidar 6
6.4 Lancement de la projection 7
6.5 Utilisation d'un rosbag à la place du lidar 7

7 Conclusion et prochaine étape **7**

1 Objectif et chaîne de traitement

Le lidar Unitree L2 publie les acquisitions sous la forme de messages ROS 2 `sensor_msgs/msg/PointCloud2` sur le topic `/unilidar/cloud`. Un nuage contient typiquement environ 5 000 à 5 400 points. Les champs disponibles comprennent notamment `x`, `y`, `z`, `intensity`, `ring` et `time`. La version simple du programme utilise d'abord uniquement les coordonnées cartésiennes (x, y, z).

La chaîne actuellement mise en place est la suivante :

`PointCloud2` $\longrightarrow$ décodage XYZ $\longrightarrow$ projection sphérique $\longrightarrow$ image de profondeur $\longrightarrow$ affichage et sauvegarde `.npy`.

Le lidar et la scène étant supposés immobiles pendant les essais, plusieurs acquisitions successives peuvent être fusionnées sans recalage géométrique.

2 Références méthodologiques

L'étude a commencé par l'article de Roveri et al. [**roveri2018**], qui montre comment convertir automatiquement un nuage de points en images de profondeur afin d'exploiter des réseaux convolutifs 2D. La réalisation choisie ici suit plus précisément la projection sphérique de RangeNet++ proposée par Milioto et al. [**milioto2019**]. Cette représentation transforme chaque direction 3D en une position (u, v) dans une image panoramique.

L'objectif actuel n'est pas encore de reproduire l'ensemble du réseau RangeNet++, mais de valider son étape de mise en image sur les données du Unitree L2.

3 Projection sphérique

3.1 Coordonnées sphériques

Pour un point $\mathbf{p} = (x, y, z)$, la distance radiale est

$$r = \sqrt{x^2 + y^2 + z^2}. \tag{1}$$

Les angles horizontal et vertical utilisés par l'implémentation sont

$$\psi = -\operatorname{atan2}(y, x), \tag{2}$$

$$\phi = \arcsin\left(\frac{z}{r}\right). \tag{3}$$

Le signe négatif est inclus dans la définition de ψ. Par conséquent, l'expression $\psi/\pi + 1$ utilisée dans le code est algébriquement équivalente à l'écriture $1 - \operatorname{atan2}(y, x)/\pi$ de RangeNet++.

3.2 Passage aux pixels

Pour une image de hauteur H et de largeur W, les indices sont calculés par

$$u = \left\lfloor \frac{1}{2}\left(\frac{\psi}{\pi} + 1\right) W \right\rfloor, \tag{4}$$

$$v = \left\lfloor \left(1 - \frac{\phi - \phi_{\min}}{\phi_{\max} - \phi_{\min}}\right) H \right\rfloor. \tag{5}$$

Dans le programme, u correspond à la colonne et v à la ligne. Les angles *yaw* et *pitch* ne sont donc pas les valeurs de l'image : ils servent à déterminer la position du point. Le pixel contient la distance

$$I(v, u) = r. \tag{6}$$

Lorsqu'un pixel reçoit plusieurs points, la distance la plus faible est conservée. Un pixel sans mesure reçoit la valeur `NaN` et apparaît en noir dans l'affichage.

3.3 Champ de vision

Le mode standard du L2 couvre 360° horizontalement et 90° verticalement [**unitreeSDK**]. Les paramètres retenus sont donc

$$\phi_{\min} = 0^\circ, \qquad \phi_{\max} = 90^\circ. \tag{7}$$

Une plage 0°–30° ne constituerait pas une nouvelle calibration : elle supprimerait volontairement la partie haute du champ de vision.

4 Premiers résultats

4.1 Projection d'un seul nuage

Une première image de taille 160×512 contient 81 920 pixels, alors qu'un nuage n'apporte qu'environ 5 200 mesures. Même sans collision, le taux de remplissage maximal est donc voisin de

$$\frac{5\,200}{160 \times 512} \times 100 \simeq 6.35\,\%. \tag{8}$$

Cette résolution produit nécessairement une image très creuse. Une grille 72×72, soit 5 184 pixels, a ensuite permis de rendre la scène et la silhouette observables. La première capture présente environ 2 350 pixels valides, soit approximativement 45 % de remplissage. Elle montre cependant une forte pixellisation et plusieurs bandes noires verticales (figure 1).

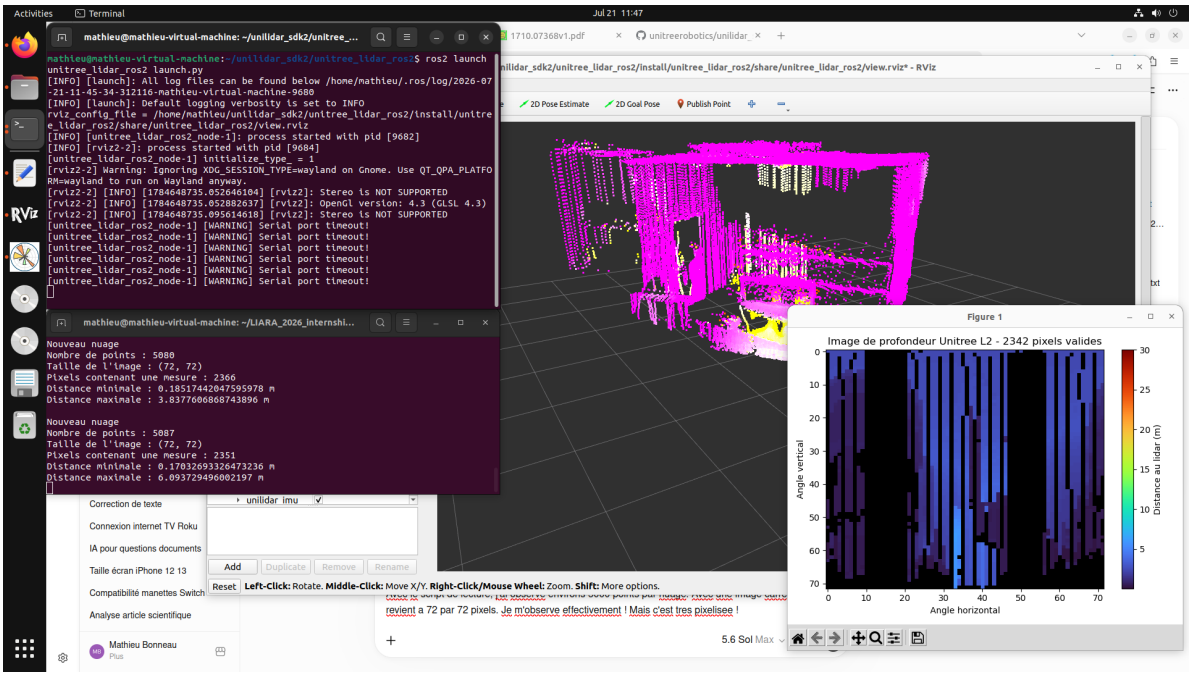


FIGURE 1 – Première projection sphérique en 72×72. La scène devient reconnaissable, mais un seul nuage laisse des bandes angulaires non échantillonnées.

4.2 Origine des bandes noires

Le choix $\sqrt{5\,200} \approx 72$ suppose implicitement une répartition uniforme des points dans l'image. Cette hypothèse n'est pas respectée : le L2 acquiert progressivement les directions selon son motif de balayage. Plusieurs points tombent dans une même cellule (u, v), tandis que certaines cellules ne sont pas visitées pendant un message `PointCloud2`.

Les bandes ne signalent donc pas une erreur dans les équations. Elles représentent principalement des directions qui seront mesurées dans les acquisitions suivantes.

5 Améliorations apportées

5.1 Accumulation de plusieurs acquisitions

Une fenêtre glissante a d'abord été utilisée pour concaténer quatre nuages. Dans l'image 72×72, le taux de remplissage mesuré sur le bag de test évolue approximativement comme suit :

Nombre de nuages	Pixels remplis
1	49 %
2	77 %
3	89 %
4	96 %

Sur la seconde capture, 4 699 pixels sur 5 184 sont valides, soit 90.6 %. Les grandes bandes noires ont disparu et les formes sont beaucoup plus continues (figure 2).

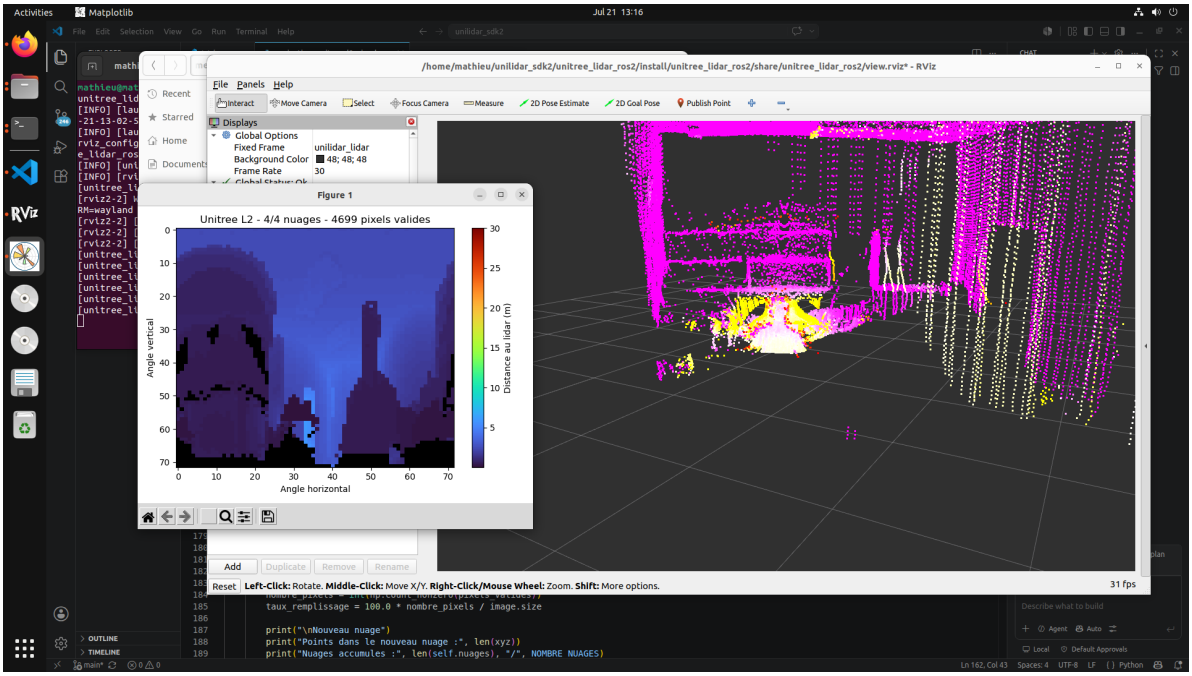


FIGURE 2 – Projection améliorée après accumulation de quatre nuages. Les bandes verticales ont presque disparu et 4 699 pixels sont valides.

5.2 Choix d'une image panoramique

Une image carrée ne respecte pas la différence entre les deux champs de vue. Pour obtenir la même résolution angulaire horizontalement et verticalement, il faut imposer

$$\frac{W}{H} = \frac{360^\circ}{90^\circ} = 4. \tag{9}$$

Une dimension entière a été retenues comme référence :

— sous 5 200 pixels : $H = 36$, $W = 144$, soit 5 184 pixels ;

Le tableau 1 rassemble les taux moyens mesurés sur le bag fourni. Les valeurs dépendent de la scène, mais elles montrent le compromis entre résolution spatiale et densité.

TABLE 1 – Comparaison de plusieurs dimensions panoramiques.

Dimension $H \times W$	Pixels	4 acquisitions	8 acquisitions
31×124	3 844	84.0 %	97.9 %
36×144	5 184	76.4 %	94.7 %
48×192	9 216	61.6 %	84.6 %
64×256	16 384	48.4 %	71.8 %

5.3 Fusion optimisée des images

La première version recalculait la projection de tous les points concaténés. Avec huit nuages, elle traitait donc environ 40 000 points à chaque callback. La version optimisée projette uniquement le nouveau nuage, puis conserve les dernières images dans un `deque`. La fusion est effectuée pixel par pixel :

$$I_{\text{fusion}}(v,u) = \min_{k \in \mathcal{K}_{v,u}} I_k(v,u), \tag{10}$$

où $\mathcal{K}_{v,u}$ désigne les images possédant une mesure valide au pixel (v,u). En Python, cette opération est réalisée par `numpy.fmin.reduce`, qui ignore les `NaN`. Cette méthode a été vérifiée comme strictement équivalente à la projection des nuages concaténés.

Avec huit acquisitions, la phase initiale couvre environ 0.67 seconde à une fréquence proche de 12 nuages par seconde. Une fois le tampon rempli, une nouvelle image reste produite à chaque message. L'affichage Matplotlib peut être limité à 5 images par seconde sans réduire la fréquence du futur modèle.

6 Commandes nécessaires

6.1 Dépendances Python

Dans un environnement ROS 2 où `rclpy` et `sensor_msgs_py` sont déjà installés :

```
python3 -m pip install numpy matplotlib
```

6.2 Compilation du paquet ROS 2 Unitree

À adapter à la distribution ROS 2 installée :

```
source /opt/ros/<distribution>/setup.bash
cd ~/unilidar_sdk2/unitree_lidar_ros2
colcon build --symlink-install
source install/setup.bash
```

6.3 Démarrage du lidar

Dans un premier terminal :

```
source /opt/ros/<distribution>/setup.bash
cd ~/unilidar_sdk2/unitree_lidar_ros2
source install/setup.bash
ros2 launch unitree_lidar_ros2 launch.py
```

Il est possible de vérifier le topic et sa fréquence avec :

```
ros2 topic list
ros2 topic info /unilidar/cloud
ros2 topic hz /unilidar/cloud
```

6.4 Lancement de la projection

Dans un second terminal :

```
source /opt/ros/<distribution>/setup.bash
cd ~/unilidar_sdk2/unitree_lidar_ros2
source install/setup.bash
cd /chemin/vers/le/script
python3 projection_unitree_l2_simple.py
```

Le fichier `image_profondeur_l2.npy` est enregistré dans le dossier depuis lequel le script est lancé.

6.5 Utilisation d'un rosbag à la place du lidar

Dans un terminal ROS 2 :

```
ros2 bag info /chemin/vers/le/dossier_du_bag
ros2 bag play /chemin/vers/le/dossier_du_bag
```

Le script de projection est lancé simultanément dans un autre terminal.

7 Conclusion et prochaine étape

La projection sphérique du Unitree L2 est correctement mise en œuvre. Les angles *yaw* et *pitch* déterminent les pixels (u, v), tandis que l'image conserve la distance radiale. Les bandes noires observées avec un seul nuage proviennent du motif de balayage et non des équations. L'accumulation de plusieurs acquisitions augmente fortement la densité, et la fusion de petites images déjà projetées permet de limiter le coût de calcul.

Pour la suite orientée deep learning, l'image de distance devra être complétée par les canaux x, y, z, l'intensité et un masque de validité. Le nombre d'acquisitions devra être choisi selon le compromis entre densité et fraîcheur temporelle.